Simulation and Machine Learning for Finance

1 Numerical Foundations

1.1 Norms and Errors

Vector Norms: for $x \in \mathbb{R}^n$,

$$\|x\|_1 = \sum_i |x_i|, \qquad \|x\|_2 = \Big(\sum_i x_i^2\Big)^{1/2} = \sqrt{x^T x}, \qquad \|x\|_\infty = \max_i |x_i|$$

all special cases of $\|x\|_p = (\sum_i |x_i|^p)^{1/p}$. An unlabelled norm means the 2 norm (or any norm). Applied to a vector of pointwise errors e, $\|e\|_\infty$ is the worst case error, while $\|e\|_1/n$ and $\|e\|_2/\sqrt{n}$ are average (mean absolute and root mean square) errors.

Absolute and Relative Error:

$$E_{abs} = \|x - x_{approx}\|, \qquad E_{rel} = \frac{\|x - x_{approx}\|}{\|x\|}$$

Relative error is the fair way to compare accuracy across quantities of very different size.

Sources of Error: *roundoff*, from finite precision arithmetic, and *algorithmic* (truncation, discretisation, sampling) errors that remain even with exact arithmetic.

1.2 Floating Point Arithmetic

A double is stored as $f = x \times 2^n$ with a 52 bit mantissa x, about 16 significant decimal digits. Every operation carries a relative error of order **machine epsilon** $\epsilon_m \approx 2.2 \times 10^{-16}$ (`np.finfo(float).eps`); integers are exact.

Catastrophic Cancellation: subtracting nearly equal numbers destroys significant digits, since the leading digits cancel and only noisy trailing digits survive. The cure is algebraic rewriting, e.g.

$$1 - \cos x = 2\sin^2(x/2), \qquad \sqrt{x+1} - \sqrt{x} = \frac{1}{\sqrt{x+1} + \sqrt{x}}$$

Apart from such cases, roundoff is rarely the limiting factor in well designed computations.

1.3 Big O Notation

$f(x) = O(g(x))$ as $x \to \infty$ if $|f(x)| \le M g(x)$ for some M and all $x \ge x_0$; as $x \to 0$, the same bound holds for $|x| < \delta$. In practice we read it off by inspection:

- **Cost** as size grows keeps the *largest* power: $C(n) = \frac{2}{3}n^3 + 2n^2 + 8n = O(n^3)$, so doubling n multiplies the cost by 8
- **Error** as a step shrinks keeps the *smallest* power: $T(h) = a_2 h^2 + a_4 h^4 + \cdots = O(h^2)$, so halving h divides the error by 4

1.4 Finite Differences

On a uniform grid $x_i = a + ih$, Taylor expansion $f(x+h) = f(x) + hf'(x) + \frac{h^2}{2}f''(x) + \frac{h^3}{6}f'''(x) + \dots$ gives

Scheme	Formula	Error
Forward	$\dfrac{f(x+h) - f(x)}{h}$	$O(h)$
Backward	$\dfrac{f(x) - f(x-h)}{h}$	$O(h)$
Centred	$\dfrac{f(x+h) - f(x-h)}{2h}$	$O(h^2)$
Second derivative	$\dfrac{f(x-h) - 2f(x) + f(x+h)}{h^2}$	$O(h^2)$

A scheme has **order** p if its error is $O(h^p)$. In the centred difference the even terms cancel, which is where the extra order comes from; one sided schemes gain an order wherever $f'' = 0$. On a log log plot of error against h, the slope equals the order. Relative error has the same order as absolute error as long as the true derivative is nonzero.

Two Competing Errors: truncation error falls like h^p, but roundoff in the numerator grows like ϵ_m/h. Total error is therefore minimised at a moderate h (roughly $\sqrt{\epsilon_m} \approx 10^{-8}$ for forward differences); taking h ever smaller eventually makes things worse.

1.5 Random Numbers

Monte Carlo is driven by **pseudorandom** numbers: deterministic sequences that mimic randomness. A **linear congruential generator** illustrates the idea:

$$x_{n+1} = a x_n \bmod m, \qquad u_{n+1} = x_{n+1}/m \in (0,1)$$

started from a **seed** x_0. The same seed reproduces the same stream, which is essential for debugging and for comparing methods on identical samples. LCGs have short periods and visible lattice structure, so modern libraries use far better generators. A common rule of thumb is that the period should vastly exceed (at least the square of) the number of draws used.

Never Write Your Own: random number generators, dense linear algebra solvers and fast Fourier transforms. Use tested libraries (e.g. `rng = np.random.default_rng(seed)`).

Inverse Transform Sampling: if $Y \sim \mathcal{U}(0,1)$ and F_X is an invertible CDF, then $X = F_X^{-1}(Y)$ has distribution F_X, because $F_X(X)$ is uniform. For normals, $X = \Phi^{-1}(Y)$ (`scipy.stats.norm.ppf`); for Exp(λ), $X = -\ln(1-Y)/\lambda$.

1.6 Complexity, Conditioning and Stability

Complexity: time (or memory) cost as a function of input size. Dot product of length n vectors: $O(n)$; matrix vector product: $O(n^2)$; naive matrix product: $O(n^3)$; nested loops over n and m: $O(nm)$; a loop over $i < n$ with an inner loop over $j < i$: $O(n^2)$; an inner loop that doubles its counter: $O(n \log n)$. Library documentation usually states the complexity of learning algorithms in terms of samples and features, which matters for large data.

Condition Number: sensitivity of a *problem*, not of an algorithm. For $Ax = b$,

$$\frac{\|\Delta x\|}{\|x\|} \approx \kappa(A)\frac{\|\Delta b\|}{\|b\|}, \qquad \kappa(A) = \|A\|\,\|A^{-1}\|$$

With $\kappa = 10^k$, roughly k digits are lost: in double precision, $\kappa = 10^{12}$ leaves about 4 reliable digits. No algorithm can beat the conditioning of the problem, although reformulating the problem can.

Stability: a property of an *algorithm*. An unstable scheme amplifies small errors until the numerical solution blows up, even though it is consistent with the underlying equation.

1.7 Python Essentials

NumPy, SciPy and Matplotlib form the core scientific stack; scikit-learn and pandas are used for machine learning. Arrays are zero indexed, operations are **elementwise** by default, and vectorised code over contiguous arrays is far faster than Python loops. When in doubt, print `.shape`.

```
import numpy as np
x = np.linspace(0, np.pi, 10)   # 10 equally spaced points
u = np.arange(0, 10, 2)         # 0, 2, 4, 6, 8
y = np.exp(-x**2)               # elementwise
a = np.zeros((2, 3))            # a.shape == (2, 3)
```

A *method* is simply a function attached to an object (e.g. `rng.normal`); variables created inside a function have local scope.

---

2 Monte Carlo Methods

2.1 Geometric Brownian Motion

Under the risk neutral measure the asset follows $\mathrm{d}S_t = rS_t\,\mathrm{d}t + \sigma S_t\,\mathrm{d}W_t$ with constant r and σ, so

$$S_T = S_0 \exp\Big(\big(r - \tfrac{1}{2}\sigma^2\big)T + \sigma\sqrt{T}X\Big), \qquad X \sim \mathcal{N}(0,1)$$

equivalently with σW_T, $W_T \sim \mathcal{N}(0,T)$, or with $X = \Phi^{-1}(Y)$, $Y \sim \mathcal{U}(0,1)$. S_T is log normal with

$$\mathbb{E}[S_T] = S_0 e^{rT}, \qquad \mathrm{var}[S_T] = S_0^2 e^{2rT}\big(e^{\sigma^2 T} - 1\big)$$

Variance Facts: $\mathrm{var}[X + c] = \mathrm{var}[X]$, $\mathrm{var}[cX] = c^2\mathrm{var}[X]$, $\mathrm{var}[X+Y] = \mathrm{var}[X] + 2\mathrm{cov}[X,Y] + \mathrm{var}[Y]$, $\mathrm{cov}[X,Y] = \mathbb{E}[XY] - \mathbb{E}X\,\mathbb{E}Y$, $\mathrm{corr} = \mathrm{cov}/(\sigma_X\sigma_Y)$. For a random vector, $\Sigma = \mathbb{E}[(X - \mathbb{E}X)(X - \mathbb{E}X)^T]$ is symmetric positive semi definite and $\mathbb{E}[LX] = L\,\mathbb{E}[X]$.

2.2 Monte Carlo Integration

Any integral can be read as an expectation. For $Y \sim \mathcal{U}(0,1)$,

$$\bar{f} = \int_0^1 f(y)\,\mathrm{d}y = \mathbb{E}[f(Y)] \approx \bar{f}_N = \frac{1}{N}\sum_{i=1}^N f(Y_i)$$

$\bar{f}_N$ is itself random: it is **unbiased** ($\mathbb{E}\bar{f}_N = \bar{f}$) and **consistent** (law of large numbers). Integration over the whole line against the normal density is the same idea after the change of variable $y = \Phi(x)$: $\int_0^1 f(y)\,\mathrm{d}y = \int_{-\infty}^{\infty} f(\Phi(x))\phi(x)\,\mathrm{d}x$, so in practice we simply average $g(X_i)$ over $X_i \sim \mathcal{N}(0,1)$.

Error via the CLT: for large N, $\bar{f}_N - \bar{f} \approx \mathcal{N}(0, \sigma_f^2/N)$, so with 95% confidence

$$|\bar{f}_N - \bar{f}| \le 1.96\frac{\sigma_f}{\sqrt{N}}, \qquad \hat{\sigma}_f^2 = \frac{1}{N-1}\sum_{i=1}^N \big(f(Y_i) - \bar{f}_N\big)^2$$

$\sigma_f/\sqrt{N}$ is the **standard error of the mean**. The rule 68, 95, 99.7 per cent within 1, 2, 3 standard deviations applies, and $z_{1-\alpha/2}$ replaces 1.96 at other levels.

Practical Points:

- $\hat{\sigma}_f$ only sets the error bar, so one or two significant digits suffice; a 10% error in the variance changes the interval width by only about 5%
- The same samples give both the estimate and its error bar
- To reach accuracy ϵ with 95% confidence we need $N \approx (1.96\sigma_f/\epsilon)^2$; e.g. $\sigma_f = 15$ and $\epsilon = 0.1$ require about 86,000 samples

Convergence Rate: the error is $O(N^{-1/2})$: four times the work halves the error, a hundred times the work gains one digit. A grid based trapezoidal rule converges as $O(N^{-2})$ in one dimension, but grid methods degrade rapidly with dimension (curse of dimensionality), whereas Monte Carlo keeps $O(N^{-1/2})$ in any dimension and handles problems with no grid formulation. This is why Monte Carlo dominates for path dependent and multi asset derivatives.

2.3 Pricing European Options

The price is the expected discounted payoff, $V = \mathbb{E}[f(S_T)]$ with $f = e^{-rT}(S_T - K)^+$ for a call and $e^{-rT}(K - S_T)^+$ for a put.

```
rng = np.random.default_rng(2024)
x = rng.normal(0, 1, n)
s_t = s0*np.exp((r - 0.5*sigma**2)*t + sigma*np.sqrt(t)*x)
f = np.exp(-r*t)*np.maximum(s_t - k, 0)
price = f.mean()
half_width = 1.96*f.std(ddof=1)/np.sqrt(n)
```

Different seeds give different estimates; always report the confidence interval and compare with Black Scholes where a closed form exists. Other payoffs that depend only on S_T (spreads, digitals) need only a change of the payoff line.

2.4 Greeks

Sensitivities of $V(S,K,T,r,\sigma)$: $\Delta = \partial V/\partial S$, $\Gamma = \partial^2 V/\partial S^2$, Vega $= \partial V/\partial\sigma$, and so on. Write θ for a generic parameter.

Finite Differences with Independent Samples Fail: for the centred estimate from independent simulations,

$$\mathrm{var}\left[\frac{V_{j+1}^N - V_{j-1}^N}{2\Delta\theta}\right] = \frac{\mathrm{var}[V_{j+1}^N] + \mathrm{var}[V_{j-1}^N]}{4\Delta\theta^2} \approx \frac{\mathrm{var}[V_j^N]}{2\Delta\theta^2}$$

which **diverges** as $\Delta\theta \to 0$ for fixed N (the approximation assumes the variance changes little across nearby parameters).

Path Recycling: reuse the *same* normals for $\theta \pm \Delta\theta$. The two estimates become strongly correlated, the covariance term cancels the divergence, and

$$\mathrm{var}\left[\frac{f(\theta+\Delta\theta) - f(\theta-\Delta\theta)}{2\Delta\theta}\right] \to \mathrm{var}\left[\frac{\partial f}{\partial\theta}\right]$$

independent of $\Delta\theta$. For delta: simulate S_T from $S_0 \pm \mathrm{d}S$ with common X_i and average the centred difference of payoffs.

Pathwise Derivative: differentiate inside the expectation with the chain rule. Since $\partial S_T/\partial S_0 = S_T/S_0$ and $\partial S_T/\partial\sigma = S_T(-\sigma T + \sqrt{T}X)$,

$$\Delta = \mathbb{E}\left[e^{-rT}\mathbb{1}_{S_T>K}\frac{S_T}{S_0}\right], \qquad \text{Vega} = \mathbb{E}\Big[e^{-rT}\mathbb{1}_{S_T>K}S_T\big(\sqrt{T}X - \sigma T\big)\Big]$$

(`np.heaviside` gives the indicator). Valid when the payoff is continuous and piecewise differentiable in the parameter; it fails for discontinuous payoffs such as digitals.

Likelihood Ratio: move the parameter dependence onto the density instead: $V = \int f(s)p_\theta(s)\,\mathrm{d}s$, so

$$\frac{\partial V}{\partial\theta} = \mathbb{E}\left[f(S_T)\frac{\partial \ln p_\theta(S_T)}{\partial\theta}\right], \qquad \frac{\partial \ln p}{\partial S_0} = \frac{\ln(S_T/S_0) - (r - \frac{1}{2}\sigma^2)T}{S_0\sigma^2 T} = \frac{X}{S_0\sigma\sqrt{T}}$$

using the log normal density of S_T. The payoff is never differentiated, so this works for discontinuous payoffs (digitals, barriers), at the cost of typically higher variance than the pathwise method.

---

3 Variance Reduction and Baskets

The Monte Carlo error is $\sigma_f/\sqrt{N}$. The $N^{-1/2}$ rate cannot be changed, but σ_f can, by estimating the same mean with a lower variance quantity.

3.1 Antithetic Variates

Y and $1-Y$ are both uniform (for normals, X and $-X$). With $Z = \frac{1}{2}[f(Y) + f(1-Y)]$, $\mathbb{E}Z = \mathbb{E}f$ and

$$\mathrm{var}[Z] = \tfrac{1}{2}\mathrm{var}[f] + \tfrac{1}{2}\mathrm{cov}\big[f(Y), f(1-Y)\big]$$

The factor $\frac{1}{2}$ is just a sample of two; genuine reduction requires **negative covariance**, which holds for monotone f.

- f linear: Z is constant and the variance is zero
- f monotone (e.g. call or put payoffs in X): solid gains
- f symmetric about the midpoint, e.g. $(y - \frac{1}{2})^2$ or a straddle: $f(1-Y) = f(Y)$, no gain at double the cost

For options, simulate $S_T^{\pm}$ from $\pm X_i$ and average the two payoffs per pair. It is the simplest technique and often cheaper than twice the cost in practice, because random number generation dominates and is shared.

3.2 Control Variates

Use a quantity g correlated with f whose mean $\bar{g}$ is known exactly:

$$f_c = f - c(g - \bar{g}), \qquad \mathbb{E}f_c = \mathbb{E}f, \qquad c^* = \frac{\mathrm{cov}[f,g]}{\mathrm{var}[g]}$$

giving

$$\mathrm{var}[f_c] = \mathrm{var}[f]\big(1 - \mathrm{corr}[f,g]^2\big)$$

Strong correlation of either sign gives large reductions. For a European call, $g = S_T$ works well, with $\mathbb{E}S_T = S_0e^{rT}$ and $\mathrm{var}S_T$ known for GBM; $\mathrm{cov}[f,g]$ is estimated from the same sample. Other natural controls are a geometric average Asian option for an arithmetic one, or a closed form vanilla for an exotic.

3.3 Importance Sampling

Change the sampling density to put samples where the integrand matters:

$$\mathbb{E}_{\rho_1}[f(Y)] = \int f(y)\frac{\rho_1(y)}{\rho_2(y)}\rho_2(y)\,\mathrm{d}y = \mathbb{E}_{\rho_2}\left[f(Y)\frac{\rho_1(Y)}{\rho_2(Y)}\right]$$

The ratio is the likelihood ratio (Radon Nikodym derivative). We need $\rho_2 > 0$ wherever $f \neq 0$; the aim is to make $f\rho_1/\rho_2$ as close to constant as possible.

Out of the Money Calls: most naive samples have zero payoff. With $S_T(y_K) = K$,

$$y_K = \Phi\left(\frac{\ln(K/S_0) - (r - \frac{1}{2}\sigma^2)T}{\sigma\sqrt{T}}\right)$$

sample $Y \sim \mathcal{U}(y_K, 1)$, set $X = \Phi^{-1}(Y)$, and average $(1 - y_K)f(S_T)$. For a put, sample $\mathcal{U}(0, y_K)$ and weight by y_K.

Shifted Normal Alternative: draw $X \sim \mathcal{N}(\mu, 1)$ with μ moving mass towards the strike and weight by $\phi(X)/\phi(X-\mu) = e^{-\mu X + \mu^2/2}$; no CDF inversion is needed.

3.4 Stratified Sampling

Split $[0,1]$ into M equal strata, draw $n = N/M$ uniform points in each, and average the stratum means:

$$\tilde{f}_N = \frac{1}{M}\sum_{m=0}^{M-1}\bar{f}^m, \qquad \mathrm{var}[\tilde{f}_N] = \mathrm{var}[\bar{f}_N] - \frac{1}{N}\frac{1}{M}\sum_{m=0}^{M-1}(I_m - I)^2$$

where I_m is the mean of f on stratum m. Stratification removes the between strata part of the variance, so it never increases variance and helps most when f varies a lot across strata. Combine the per stratum *variances* by averaging them (not by taking the variance of the stratum means).

3.5 Correlated Normals and Basket Options

If $X \sim \mathcal{N}(0, I)$ and $Y = LX$ then $\mathrm{cov}[Y] = LL^T$. To sample with a target covariance Σ, take its **Cholesky factor** ($\Sigma = LL^T$, L lower triangular with positive diagonal; `scipy.linalg.cholesky(..., lower=True)`).

Basket Call: $B(T) = \frac{1}{M}\sum_m S_m(T)$ with discounted payoff $e^{-rT}(B(T) - K)^+$, and each asset GBM with correlated drivers, $\mathbb{E}[\,\mathrm{d}W_i\,\mathrm{d}W_j] = \rho_{ij}\,\mathrm{d}t$. Then

$$\Sigma = \mathrm{diag}(\sigma)\,\rho\,\mathrm{diag}(\sigma), \qquad S_m(T) = S_m(0)\exp\Big(\big(r - \tfrac{1}{2}\sigma_m^2\big)T + \sqrt{T}\,Y_m\Big), \quad Y = LX$$

Algorithm: build Σ, factor it once, draw an $M \times N_{paths}$ array of normals, correlate, compute terminal prices, average across assets, apply the payoff and average across paths. No closed form exists for arithmetic baskets, so this is a natural Monte Carlo problem; higher correlation raises the basket's volatility and hence the call price.

---

4 SDE Simulation and Path Dependent Options

Closed form terminal distributions require constant parameters and payoffs depending only on S_T. Otherwise we simulate whole **sample paths** on a grid $t_n = n\Delta t$, $\Delta t = T/N_{steps}$, and average payoffs over N_{paths} paths.

4.1 Time Stepping

ODEs (Euler): for $\dot{y} = f(t,y)$, the forward difference gives $y_{n+1} = y_n + \Delta t\, f(t_n, y_n)$, with local error $O(\Delta t^2)$ per step and global error $O(\Delta t)$ (first order).

Brownian Motion: $W_{n+1} = W_n + \Delta W_n$ with $\Delta W_n = \sqrt{\Delta t}\,\xi_n$, $\xi_n \sim \mathcal{N}(0,1)$ iid.

Euler Maruyama: for $\mathrm{d}X_t = a(X_t,t)\,\mathrm{d}t + b(X_t,t)\,\mathrm{d}W_t$,

$$X_{n+1} = X_n + a(X_n, t_n)\Delta t + b(X_n, t_n)\Delta W_n$$

For GBM, $S_{n+1} = S_n(1 + r\Delta t + \sigma\Delta W_n)$. Replacing constants by $r(S,t)$ and $\sigma(S,t)$ gives **local volatility** models at no extra difficulty; random σ is **stochastic volatility**.

Milstein: adds the correction from Itô's formula,

$$X_{n+1} = X_n + a\Delta t + b\Delta W_n + \tfrac{1}{2}bb'\big(\Delta W_n^2 - \Delta t\big), \qquad b' = \partial b/\partial x$$

For GBM: $S_{n+1} = S_n\big(1 + r\Delta t + \sigma\Delta W_n + \frac{1}{2}\sigma^2(\Delta W_n^2 - \Delta t)\big)$.

Why Milstein Is More Accurate: expanding the exact GBM step $S_n\exp((r - \frac{1}{2}\sigma^2)\Delta t + \sigma\xi\sqrt{\Delta t})$ to order Δt gives $S_n\big(1 + \sigma\xi\sqrt{\Delta t} + (r + \frac{1}{2}\sigma^2(\xi^2 - 1))\Delta t\big)$. Milstein matches this to $O(\Delta t)$, whereas Euler drops the $\frac{1}{2}\sigma^2(\xi^2 - 1)\Delta t$ term.

4.2 Convergence of SDE Schemes

Weak (error of the mean): $|\mathbb{E}[X_T] - \mathbb{E}[X_N]| = O(\Delta t^p)$, about distributions and prices.

Strong (mean of the error): $\mathbb{E}\|X_T - X_N\| = O(\Delta t^p)$, pathwise, comparing paths driven by the same Brownian increments.

Scheme (GBM)	Strong order	Weak order
Euler Maruyama	1/2	1
Milstein	1	1

For pricing (expectations) the weak error is what matters, so Euler is usually adequate; strong order matters for pathwise quantities and multilevel methods.

Spurious Paths: an Euler step goes negative if $\xi_n < -(1 + r\Delta t)/(\sigma\sqrt{\Delta t})$. For GBM this is vanishingly rare and such paths can be discarded; for models like $\mathrm{d}S = rS\,\mathrm{d}t + S^\gamma\,\mathrm{d}W$ with $\gamma < \frac{1}{2}$ zero is genuinely reachable, and paths are typically absorbed at zero.

4.3 Balancing Paths and Steps

With V the true price and $\bar{V}$ the estimate, Euler gives

$$\mathrm{MSE} = \mathbb{E}[(\bar{V} - V)^2] \approx \frac{c_1}{N_{paths}} + \frac{c_2}{N_{steps}^2}$$

(sampling variance plus squared weak bias). For a fixed budget $C = N_{paths}N_{steps}$, minimising gives

$$N_{steps} = \left(\frac{2c_2C}{c_1}\right)^{1/3}, \qquad \frac{c_1}{N_{paths}} = 2\,\frac{c_2}{N_{steps}^2}$$

At the optimum the sampling variance is **twice** the squared bias, a rule of thumb that needs no knowledge of the constants. In practice: halving Δt (halving the bias) justifies roughly four times as many paths.

4.4 Path Dependent Options

Simulate an $(N_{steps}+1) \times N_{paths}$ array of prices with daily steps ($\Delta t = T/260$ trading days, say) and evaluate the payoff on each column.

- **Asian call:** $e^{-rT}\big(\frac{1}{N}\sum_n S_n - K\big)^+$; averaging makes it cheaper than a vanilla and harder to manipulate at expiry
- **Floating strike lookback call:** $e^{-rT}(S_T - \min_t S_t)$ via `np.amin(s, axis=0)`
- **Up and out call:** $e^{-rT}(S_T - K)^+\mathbb{1}\{\max_t S_t < S_b\}$ via `np.heaviside(sb - s.max(axis=0), 0)`

Antithetic paths come for free: step one array with $+\Delta W_n$ and a twin with $-\Delta W_n$.

Continuous Monitoring Bias: checking the barrier only at grid points misses crossings between them, overpricing knock out options. For a down barrier S_b with both endpoints above it, the Brownian bridge crossing probability on a step (applied to log prices) is

$$P_n = \exp\left(-\frac{2\ln(S_n/S_b)\ln(S_{n+1}/S_b)}{\sigma^2\Delta t}\right)$$

and the payoff is weighted by $\prod_n(1 - P_n)$. Alternatively, sample the within step minimum of $x = \ln S$ as $m_n = \frac{1}{2}\big(x_n + x_{n+1} - \sqrt{(x_{n+1} - x_n)^2 - 2\sigma^2\Delta t\ln U_n}\big)$ with $U_n \sim \mathcal{U}(0,1)$ and knock out if any m_n falls below $\ln S_b$.

4.5 Brownian Bridge

A Brownian motion pinned at both ends: given B_0 and B_T,

$$\mathbb{E}[B_t] = B_0 + \frac{t}{T}(B_T - B_0), \qquad \mathrm{var}[B_t] = \frac{t(T-t)}{T}$$

largest at the midpoint, $T/4$, half the variance of a free path at $T/2$.

Construction: recursively fill midpoints, $B_{T/2} = \frac{1}{2}(B_0 + B_T) + \frac{\sqrt{T}}{2}X$, then the midpoints of each half, and so on. Equivalently, simulate a free path W and set $B_t = B_0 + \frac{t}{T}(B_T - B_0) + W_t - \frac{t}{T}W_T$.

Use in Stratification: draw the *terminal* value first, stratified ($W_T = \sqrt{T}\Phi^{-1}(Y)$ with Y in stratum m), then fill in the path with a bridge. The variable that matters most for the payoff is stratified while paths keep the correct law.

4.6 Beyond Euler (Deterministic Problems)

Accuracy Concepts: local truncation error $O(\Delta t^{k+1})$ means order k, with global error $O(\Delta t^k)$ over a fixed horizon (N steps times the one step error). A method is **consistent** if the truncation error divided by Δt vanishes as $\Delta t \to 0$.

Higher Order Schemes: *midpoint* ($\tilde{y} = y_n + \frac{\Delta t}{2}f(y_n)$, $y_{n+1} = y_n + \Delta t f(\tilde{y})$) and explicit *trapezoidal* or Heun ($y_{n+1} = y_n + \frac{\Delta t}{2}[f(y_n) + f(y_n + \Delta t f(y_n))]$) are second order at two function evaluations per step; classical **RK4** is fourth order at four evaluations,

$$y_{n+1} = y_n + \tfrac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4)$$

with $k_1 = \Delta t f(t_n, y_n)$, $k_2 = \Delta t f(t_n + \frac{\Delta t}{2}, y_n + \frac{k_1}{2})$, $k_3 = \Delta t f(t_n + \frac{\Delta t}{2}, y_n + \frac{k_2}{2})$, $k_4 = \Delta t f(t_n + \Delta t, y_n + k_3)$.

Heat Equation: for $u_t = u_{xx}$ with Dirichlet boundaries, forward Euler in time and centred differences in space give

$$U_j^{n+1} = U_j^n + \nu\big(U_{j-1}^n - 2U_j^n + U_{j+1}^n\big), \qquad \nu = \frac{\Delta t}{\Delta x^2}$$

an explicit scheme that is stable only if $\nu \le \frac{1}{2}$. Pricing PDEs such as Black Scholes reduce to this type of equation after a change of variables.

---

5 Machine Learning Fundamentals

5.1 Setting

Machine learning lets a computer learn a task from data rather than from explicit rules.

- **Supervised:** labelled pairs (x_i, y_i); learn f with $y \approx f(x)$, or more generally $p(y \mid x)$
- **Unsupervised:** unlabelled x_i; discover structure, or learn $p(x)$ (clustering, dimension reduction such as PCA via the SVD)
- **Reinforcement:** learn actions from rewards received while interacting with an environment
- **Self supervised** and **generative** models create their own labels or learn to sample new data

The main supervised tasks are **regression** (real valued targets) and **classification** (one of k classes).

Notation: each example $x_i \in \mathbb{R}^n$ is a row of the **design matrix** $X \in \mathbb{R}^{m \times n}$ (rows are examples, columns are features); targets form the vector y.

Workflow (scikit-learn): load and inspect data; clean, encode and scale; split into training and test sets; choose an estimator and its hyperparameters; `fit`; `predict`; evaluate. The test set is a *random* subset held back entirely until the end (for time series, keep chronological order instead).

5.2 Linear Regression

Model $\hat{y} = w^T x + b$; the bias is absorbed by appending a constant feature. Minimising the training MSE

$$\mathrm{MSE}_{train} = \frac{1}{m}\|Xw - y\|_2^2 \;\Rightarrow\; \nabla_w \mathrm{MSE} = 0 \;\Rightarrow\; w = (X^TX)^{-1}X^Ty$$

the **normal equations**. $(X^TX)^{-1}X^T$ is the Moore Penrose pseudo inverse (for full column rank): Xw is the projection of y onto the column space of X. **Polynomial regression** ($\hat{y} = b + \sum_k w_k x^k$) is still linear in w, so the same machinery fits it.

5.3 Capacity, Overfitting and Regularisation

We want small training error *and* a small gap between training and test error.

- **Underfitting:** training error stays large; the model lacks capacity
- **Overfitting:** training error is small but test error is much larger; the model memorises noise

Test error is U shaped in capacity (e.g. polynomial degree), with an optimal capacity between the two regimes.

Regularisation: any change that aims to reduce generalisation error rather than training error. For linear models, penalise the weights:

$$J(w) = \mathrm{MSE}_{train} + \alpha\|w\|_2^2 \;\text{(ridge)}, \qquad J(w) = \mathrm{MSE}_{train} + \alpha\|w\|_1 \;\text{(lasso)}$$

α is a **hyperparameter**, not a fitted parameter.

Why Lasso Gives Sparsity: the ℓ_1 ball has corners on the axes, so the constrained optimum often lands where some weights are exactly zero. In one dimension, minimising $(w-a)^2 + \lambda|w|$ gives the **soft threshold** $w^* = \mathrm{sign}(a)\max(|a| - \lambda/2, 0)$: coefficients smaller than $\lambda/2$ are set to zero, larger ones are shrunk. This performs automatic feature selection. Since the penalty treats all weights alike, **features must be scaled** first, otherwise a feature's units decide how strongly it is penalised.

5.4 Hyperparameters and Cross Validation

Hyperparameters (regularisation strength, depth, kernel width, network size) are fixed during training and chosen by a nested procedure:

1. Split off a test set
2. On the training set, run k **fold cross validation**: train on $k-1$ folds, validate on the remaining fold, rotate, and record the mean and standard deviation of the score
3. Search hyperparameters by **grid** or **random search**, often on a logarithmic scale; each candidate costs a full cross validation
4. Refit the best configuration on all training data and report its score once on the test set

Random search explores many more values of the important hyperparameters for the same budget when only a few of them matter.

5.5 Preparing Features

Scaling: standardise features (zero mean, unit variance) for distance, margin and gradient based learners; fit the scaler on training data only. Trees do not need scaling.

Categorical Data: *ordinal* categories with a natural order (education level) can be mapped to integers. *Nominal* categories (country, colour) need **one hot** encoding, one binary column per category, or **dummy** encoding, which drops one column to avoid perfect collinearity with the intercept (the dummy variable trap).

Imbalanced Classes: use stratified splits and stratified k folds so each fold keeps the class proportions; resample (undersample the majority, oversample the minority, or synthesise minority points by interpolation as in SMOTE); or reweight the loss with class weights inversely proportional to class frequency.

5.6 Performance Measures

Regression: MSE, RMSE (same units as y, sensitive to large errors), MAE (robust), and R^2 (share of variance explained).

Classification: from the confusion matrix (TP, TN, FP, FN):

$$\text{Accuracy} = \frac{TP+TN}{\text{all}}, \qquad \text{Precision} = \frac{TP}{TP+FP}, \qquad \text{Recall} = \frac{TP}{TP+FN}$$

$$F_\beta = (1+\beta^2)\frac{\text{Precision}\cdot\text{Recall}}{\beta^2\,\text{Precision} + \text{Recall}}, \qquad F_1 = \frac{2PR}{P+R}$$

- **Accuracy** misleads with imbalanced classes: predicting the majority class for every example can look excellent
- **Precision** matters when false positives are costly (flagging genuine payments as fraud); **recall** when false negatives are costly (missing actual fraud or defaults)
- $\beta > 1$ weights recall more, $\beta < 1$ weights precision more
- **ROC curve:** true positive rate against false positive rate $FP/(FP+TN)$ as the decision threshold varies; **AUC** is 1 for a perfect ranking and 0.5 for random guessing, and the curve helps choose a threshold with an acceptable false alarm rate

---

6 Optimisation and Classifiers

6.1 Optimisation Basics

Terminology: a *loss* measures the penalty on one example (e.g. $(\hat{y}-y)^2$); a *cost* aggregates losses over the training set, possibly with a regulariser; an *objective* is anything being optimised.

Minima: at a smooth interior minimum $\nabla L = 0$. A convex loss, $L(tx_1 + (1-t)x_2) \le tL(x_1) + (1-t)L(x_2)$, has every local minimum global (unique if strictly convex). Neural network losses are non convex, with many local minima and saddles. Example: $L(x,y) = x^2 + y^2 + xy - y$ has $\nabla L = 0$ at $2x + y = 0$, $2y + x = 1$, i.e. $(x,y) = (-\frac{1}{3}, \frac{2}{3})$, its global minimum since the quadratic part is positive definite.

Constrained Problems and Duality: for $\min L(x)$ subject to $g(x) \le 0$, form the Lagrangian $\mathcal{L}(x,\lambda) = L(x) + \lambda g(x)$ with $\lambda \ge 0$. Minimising over the primal variables gives the **dual function** $D(\lambda)$, and the **dual problem** is $\max_{\lambda \ge 0} D(\lambda)$, which for convex problems has the same optimal value. Example: minimise $\frac{1}{2}(x^2+y^2)$ subject to $x + y \ge 2$. With $g = 2 - x - y$, stationarity gives $x = y = \lambda$, so $D(\lambda) = 2\lambda - \lambda^2$, maximised at $\lambda = 1$, recovering $x = y = 1$. The dual has fewer unknowns and simpler constraints.

6.2 Support Vector Machines

Labels $y \in \{-1, +1\}$ and decision function $f(x) = w^Tx + b$; predict the sign of f. The set $f = 0$ is a separating hyperplane.

Maximum Margin: among separating hyperplanes choose the widest "street". Scaling so that $f = \pm 1$ on the margin edges, the margin width is $2/\|w\|$, so

$$\min_{w,b} \tfrac{1}{2}\|w\|^2 \quad \text{s.t.} \quad y_i(w^Tx_i + b) \ge 1 \;\; \forall i$$

Points on the margin edges are the **support vectors**; only they determine the solution.

Soft Margin: for nearly separable data allow slack $\xi_i \ge 0$:

$$\min \tfrac{1}{2}\|w\|^2 + C\sum_i \xi_i \quad \text{s.t.} \quad y_i(w^Tx_i + b) \ge 1 - \xi_i$$

Large C punishes violations (narrow margin, risk of overfitting); small C tolerates them (wide margin, more regularisation).

Dual Problem: with multipliers α_i,

$$\max_\alpha \sum_i \alpha_i - \frac{1}{2}\sum_{i,j} y_iy_j\alpha_i\alpha_j\langle x_i, x_j\rangle, \qquad \sum_i y_i\alpha_i = 0, \;\; 0 \le \alpha_i \le C$$

The weights have disappeared and the data enter *only through inner products.*

Kernel Trick: replace $\langle x_i, x_j\rangle$ by $k(x_i, x_j) = \langle\phi(x_i), \phi(x_j)\rangle$, which amounts to a linear classifier in a transformed feature space without ever computing ϕ. Common kernels: linear, polynomial, and the default **RBF**

$$k(x_i, x_j) = \exp\big(-\gamma\|x_i - x_j\|^2\big)$$

where larger γ gives more local, wigglier boundaries. Tune C and γ jointly on a logarithmic grid, and scale features first. Training cost grows between quadratically and cubically in the number of samples, so kernel SVMs suit small to medium datasets.

6.3 Logistic Regression

Despite its name, a classifier. A linear score (the **logit**) $z = w^Tx + b$ is mapped to a probability by the sigmoid,

$$p(y = 1 \mid x) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

and we predict class 1 when $\sigma(z) \ge \frac{1}{2}$ (i.e. $z \ge 0$), so the decision boundary is linear. Weights minimise the convex **log loss** (cross entropy)

$$-\frac{1}{m}\sum_i \big[y_i \ln p_i + (1-y_i)\ln(1-p_i)\big]$$

usually with an ℓ_2 penalty. Its strengths are calibrated probabilities, interpretability (w_j is the change in log odds per unit of feature j) and speed; the threshold can be moved away from $\frac{1}{2}$ to trade precision against recall.

---

7 Neural Networks

7.1 The Multilayer Perceptron

Neuron: a weighted sum followed by a nonlinear activation, $z = w^Tx + b$, $a = h(z)$.

Activations: Heaviside (the original perceptron, a hard threshold), sigmoid $1/(1+e^{-z})$, tanh, and **ReLU** $\max(0, z)$, the usual default (cheap, and its gradient does not vanish for active units; inactive units have zero gradient). Nonlinearity is essential: compositions of linear maps are linear, so without it depth adds nothing.

Layers: stacking neurons into layers with weight matrix $W^{(k)}$ and bias vector $b^{(k)}$,

$$a^{(0)} = x, \qquad a^{(k)} = h(W^{(k)}a^{(k-1)} + b^{(k)})$$

$$\hat{y} = \mathrm{output}\big(W^{(L)}a^{(L-1)} + b^{(L)}\big)$$

An MLP is **feedforward** (no cycles); "multilayer" refers to neurons organised in layers, not necessarily many hidden layers. **Width** is neurons per layer and **depth** the number of layers.

Output Layer: for classification, one output per class giving class probabilities (softmax; a single sigmoid unit for binary problems). For regression, the output is the identity, with *no* activation applied. Switching between the two changes the output activation and loss, not the hidden architecture.

Counting Parameters: a fully connected layer from n_{in} to n_{out} units has $(n_{in}+1)n_{out}$ parameters. For example, 3 features, two hidden layers of 4 units and one output have $(3+1)4+(4+1)4+(4+1)1=41$ weights and biases.

Other architectures (convolutional networks for images and patterns, recurrent networks for sequences) share the same training principles.

7.2 Training by Gradient Descent

All weights and biases are chosen to minimise a non convex loss L (MSE for regression, log loss for classification) plus a penalty $\alpha\|w\|^2$.

Gradient Descent: $w \leftarrow w - \eta\nabla_w L$ with learning rate η; too large diverges or oscillates, too small crawls.

Stochastic and Mini Batch: the full gradient over all examples is expensive. Shuffle the data each **epoch** (one pass through all examples) and update on mini batches of size m (typically 32 to 128):

$$w \leftarrow w - \eta\nabla_w \bar{L}_j, \qquad \bar{L}_j = \frac{1}{m}\sum_{i\in\text{batch } j} L_i$$

Batch size equal to the whole set is batch gradient descent; size one is pure SGD. The noise of small batches is cheap per step and can help escape poor local minima.

Momentum: plain descent zigzags across narrow valleys (ill conditioned problems). A velocity term smooths the path:

$$v \leftarrow \alpha v - \eta\nabla_w \bar{L}(w), \qquad w \leftarrow w + v, \qquad 0<\alpha<1$$

like a heavy ball with friction. **Nesterov momentum** evaluates the gradient at the look ahead point $w+\alpha v$, giving better corrections and faster convergence. Adaptive methods (e.g. Adam) also rescale steps per parameter.

Monitoring: plot the loss per epoch; use early stopping on a validation set and the penalty α against overfitting. Scale inputs, since gradient methods are sensitive to feature scales.

7.3 Backpropagation

The network is a composition of functions, so gradients follow from the chain rule applied from the output backwards, reusing quantities from the forward pass. For a toy composition, $f(x)=e^{-x^2}$ with $s=-x^2$ and $r=e^s$ is *evaluated* inside out but *differentiated* outside in: $f'=\frac{dr}{ds}\frac{ds}{dx}=-2xe^{-x^2}$.

In a Network: with $z_j^{(k)}=\sum_i w_{ij}^{(k)}a_i^{(k-1)}+b_j^{(k)}$ and $a_j^{(k)}=h(z_j^{(k)})$, define the error signal $\delta_j^{(k)}=\partial L/\partial z_j^{(k)}$. Then

$$\delta_j^{(L)}=\frac{\partial L}{\partial a_j^{(L)}}h'(z_j^{(L)}), \qquad \delta_i^{(k-1)}=h'(z_i^{(k-1)})\sum_j w_{ij}^{(k)}\delta_j^{(k)}$$

$$\frac{\partial L}{\partial w_{ij}^{(k)}}=a_i^{(k-1)}\delta_j^{(k)}, \qquad \frac{\partial L}{\partial b_j^{(k)}}=\delta_j^{(k)}$$

E.g. for squared loss on output j, $\partial L/\partial w_{ij}=2(\hat{y}_j-y_j)h'(z_j)a_i$. A weight feeding several outputs collects one such term per path. With ReLU, $h'=1$ for active units and 0 for inactive ones, so gradients only flow through active units. The cost of one backward pass is comparable to one forward pass, which is what makes training large networks feasible.

---

8 Trees and Ensembles

8.1 Decision Trees

A tree splits the feature space by a sequence of yes or no questions ("is feature j above t?"): internal nodes are tests, branches are outcomes, and leaves give predictions (majority class, or mean target for regression; CART handles both). For regression the fitted function is piecewise constant.

Gini Impurity: for class proportions p_k at a node,

$$G=1-\sum_{k=1}^{K}p_k^2$$

zero for a pure node and maximal, $1-1/K$, for evenly mixed classes (e.g. 2/3 for three balanced classes). Entropy $-\sum p_k\ln p_k$ is an alternative criterion.

Choosing a Split: for every feature and candidate threshold, compute the size weighted impurity of the children,

$$G_{split}=\frac{n_{left}}{n}G_{left}+\frac{n_{right}}{n}G_{right}$$

and take the smallest. Growing proceeds greedily, top down, which does not guarantee a globally optimal tree.

Strengths	Weaknesses
Easy to interpret and visualise	Unstable: small data changes alter the tree
Little preprocessing, no scaling	Piecewise constant, not smooth
Numerical and categorical features	Greedy, not globally optimal
Very cheap prediction	Overfits unless depth is limited or pruned

The key hyperparameter is `max_depth` (with minimum samples per leaf).

8.2 Random Forests (Bagging)

Bootstrap: resample N points *with replacement*. Each point is left out with probability $(1-1/N)^N\to e^{-1}\approx 0.368$, so a bootstrap sample contains about 63.2% of the distinct points; the rest are **out of bag** (OOB) and give a free validation estimate.

Random Forest: grow many deep trees, each on its own bootstrap sample and considering only a random subset of features at every split. Average predictions (regression) or take a majority vote (classification). Averaging reduces variance most when trees are weakly correlated, which is what the feature subsampling achieves. If each of B trees has variance σ^2 and pairwise correlation ρ, the average has variance $\rho\sigma^2+(1-\rho)\sigma^2/B$: more trees remove the second term, while decorrelation attacks the first.

Hyperparameters: `n_estimators` (more trees never overfit, only cost time), `max_depth` and minimum leaf size (smoothness), `max_features` (smaller gives more diverse trees), and the split criterion.

8.3 Gradient Boosting

Build an additive model sequentially, each new shallow tree correcting the current errors:

$$F_0=\text{constant (mean or base rate)}, \qquad F_m(x)=F_{m-1}(x)+\eta\,h_m(x)$$

where h_m is fitted to the residuals, or more generally to the negative gradient of the loss, i.e. gradient descent in function space. The learning rate η trades the number of trees against the risk of overfitting; small η with many trees usually generalises best.

Variants: *stochastic* gradient boosting fits each tree on a random subsample (without replacement), adding robustness and speed. **XGBoost**, **LightGBM** and scikit-learn's **histogram based gradient boosting** add regularisation, feature binning and efficient implementations, and are among the strongest tools for tabular data. Key settings: number of trees, learning rate, maximum depth and minimum child weight.

Bagging vs Boosting: bagging averages independent, low bias, high variance trees to cut variance; boosting adds dependent, high bias, low variance trees to cut bias.

8.4 Feature Importance

Impurity Based: tree models report the normalised total impurity reduction attributable to each feature (`feature_importances_`). It is fast but biased towards high cardinality features and computed on training data.

Permutation Importance: model agnostic. Record a baseline score, shuffle one feature's column to break its link with the target, and measure the drop in score (`permutation_importance`). Evaluate on held out data; correlated features can share or mask importance.

Exploratory Checks: before modelling, violin plots, pair plots and correlation matrices already reveal which features separate the classes.

8.5 Choosing a Learner

Learner	Good at	Watch out for
Linear / logistic	Speed, interpretability	Nonlinear structure
Kernel SVM	Complex boundaries, small data	Scaling, cost in N
MLP	Flexible, large data	Tuning, scaling, compute
Random forest	Robust default	Memory, extrapolation
Gradient boosting	Top tabular accuracy	Overfitting if untuned

Whatever the learner, the discipline is the same: a clean train, validation and test protocol, cross validated hyperparameters, and metrics suited to the problem.